Reviewer Pack — Minimal Rank of Tropicalized Symplectic Forms Bounds AC⁰ Par…

Entry d4f65b616cde · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 00:35:08 UTC

Minimal Rank of Tropicalized Symplectic Forms Bounds AC⁰ Parity Depth

Entry ID: `d4f65b616cde` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 00:35:08 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry (Symplectic Forms) **Field B** (complexity object): Complexity Theory: AC⁰ Parity Depth

Statement:

> {'main': 'For every AC⁰ circuit C computing PARITY on n inputs, the minimal rank of its associated tropicalized symplectic form ρ is such that ρ(C) = Θ(log(n)).', 'counterexample': 'There exists an AC⁰ circuit C with a tropicalized symplectic form ρ satisfying ρ(C) < log(n).', 'invariant': 'The invariant ψ(C) of the minimal rank of tropicalized symplectic forms is defined as follows: Given an AC⁰ circuit C, convert each gate and wire into a symplectic matrix, compute the tropicalization over the max-plus semiring, and then find the minimal rank ρ.'}

Rationale (proposer's reasoning):

> {'main': 'Symplectic geometry has been used to analyze algebraic structures in other branches of mathematics. If a similar connection exists between symplectic forms and computational complexity, it could provide new insights into understanding the inherent complexity of problems like PARITY.', 'connection': 'The study of tropical geometry has shown how algebraic structures can be analyzed using operations from tropical arithmetic, which might offer a new perspective on AC⁰ complexity.'}

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a8d7b4559d489d30`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across at least 90% of the 30 random seeds, the minimal rank ρ(C) of tropicalized symplectic forms for all AC⁰ circuits C computing PARITY on n inputs satisfies |ρ(C) - log(n)| ≤ 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalized symplectic form" AND AC0 parity depth" - "symplectic geometry" AND "AC0 complexity class"" - "minimal rank of tropicalization" IN title AND PARITY problem

Top relevant hits considered: - [http://arxiv.org/abs/0911.2389v2] Little Higgs model with new Z-parity and Dark matter - [http://arxiv.org/abs/1808.08575v5] Title-Guided Encoding for Keyphrase Generation - [http://arxiv.org/abs/1904.08455v3] Headline Generation: Learning from Decomposable Document Titles

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        # Find pivot
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below pivot
        for j in range(i+1, m):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    return A

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
```

```
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][k] += A[i][j] * B[j][k]
    return C

def tropicalize(circuit):
    # Placeholder function to convert circuit to symplectic form
    # This is a dummy implementation and should be replaced with actual logic
    m = len(circuit)
    n = len(circuit[0])
    A = [[Fraction(0) for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            if circuit[i][j] == 1:
                A[i][j] = Fraction(1)
    return gaussian_elimination(A)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        instances_tested = 0
        total_rank = 0

        for _ in range(5):  # Test with 5 different AC⁰ circuits per n
            # Generate a random AC⁰ circuit for computing PARITY on n inputs
            circuit = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

            # Convert the circuit to its associated symplectic form and compute the minimal rank
            T = tropicalize(circuit)
            rank = sum(1 for row in T if any(x != Fraction(0) for x in row))

            total_rank += rank
            instances_tested += 1

        avg_rank = total_rank / instances_tested
        results.append(avg_rank)

    metric_value = sum(results) / len(results)
    conjecture_holds = all(abs(r - math.log(n)) <= 3 for n, r in zip(n_values, results))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Minimal Rank of Tropicalized Symplectic Forms",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
```

```python
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_974346e7.py", line 100, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_974346e7.py", line 72, in run_trial
    T = tropicalize(circuit)
        ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_974346e7.py", line 55, in tropicalize
    return gaussian_elimination(A)
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_974346e7.py", line 30, in gaussian_elim
    factor = Fraction(A[j][i], A[i][i])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test without errors to confirm or refute the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14120
2	propose	ollama_remote	glm4:latest	0	0	11686
3	preregistration	ollama_remote	glm4:latest	0	0	6026
4	novelty	ollama_remote	glm4:latest	0	0	4658
5	novelty	ollama_remote	glm4:latest	0	0	5919
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13451
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15405
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11009
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12500
10	judge	ollama_remote	glm4:latest	0	0	32458

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 127234 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d4f65b616cde.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d4f65b616cde.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d4f65b616cde.tar.gz` (if generated)